

Containerized FastAPI Service with Reverse Proxy

Assignment 2 · Deployment report and evidence

Student **Yves Alain Iragena**

Course **CSC 410/510 — Cloud Computing**

Instructor **Richard Kelley**

Date **1 October 2026**

Repository **github.com/Alan-911/Cloud-Computing**

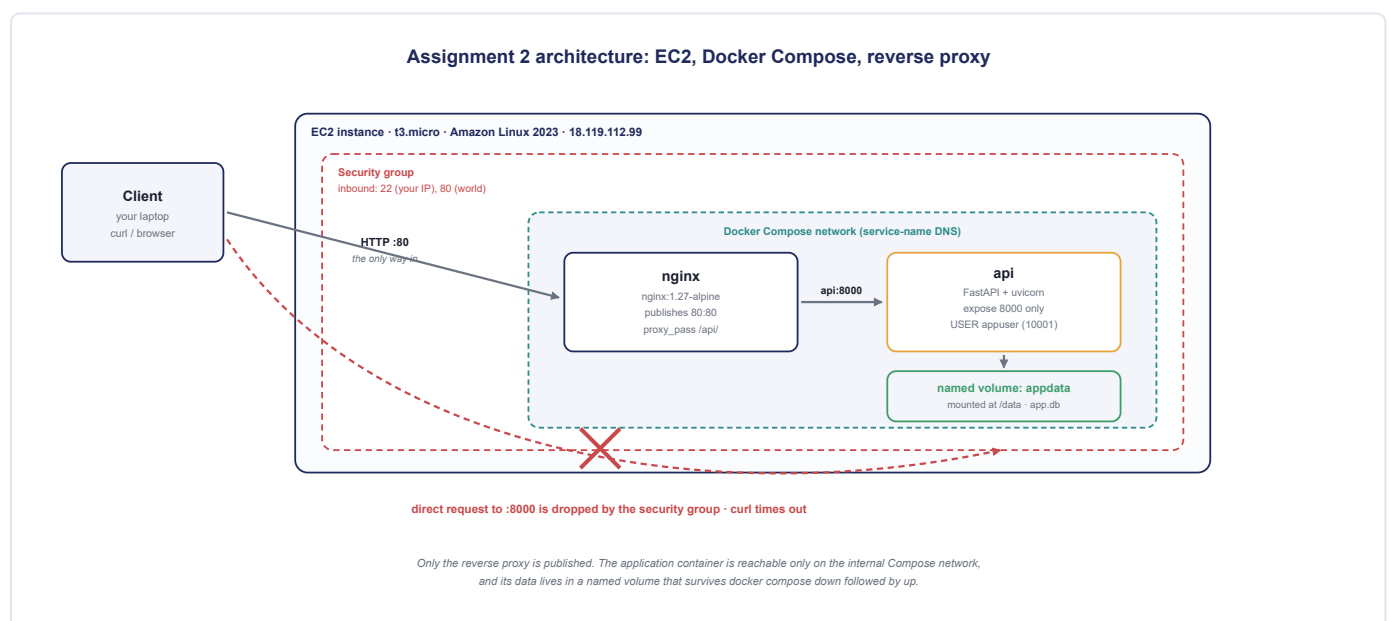
Instance **i-0fce9325be0cc0adb · t3.micro · Amazon Linux 2023 · us-east-2**

Public address **http://18.119.112.99/api/health**

This report documents a two-container system deployed to a free-tier EC2 instance. A FastAPI application runs in one container and persists data to a named Docker volume. Nginx runs in a second container as a reverse proxy and is the only service published to the host. The report records what was built, the evidence that each requirement is met, and the two problems that had to be solved along the way.

The assignment required completing `app/Dockerfile` and `docker-compose.yml`. The starter repository supplied the application, its dependencies, the nginx configuration and a working compose file.

Architecture



What was implemented

app/Dockerfile

Built from `python:3.12-slim`. Dependencies are copied and installed before the application code so that layer stays cached when only `main.py` changes. The decisive detail is the ordering around the volume:

```
RUN useradd --create-home --uid 10001 --shell /usr/sbin/nologin appuser \
    && mkdir -p /data \
    && chown -R appuser:appuser /data /app
USER appuser
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Why /data is created at build time. When Docker first creates an empty named volume it seeds the volume's ownership from whatever directory exists at that path inside the image. If /data is absent from the image, the volume is created owned by root and the non-root process cannot open the SQLite file, which fails at startup. Creating and chowning the directory during the build is what allows a non-root user and a named volume to work together.

The image also declares a `HEALTHCHECK` against `/api/health` using only the Python standard library, so `curl` never has to be installed. The command uses the exec form and omits `--reload`, so a single process runs and signals reach uvicorn directly.

docker-compose.yml

Two services and one named volume. The api service uses `expose` rather than `ports`, which keeps 8000 on the internal network and off the host. One change was made to the supplied file: nginx now waits on the API's health check rather than merely on its creation.

```
depends_on:
  api:
    condition: service_healthy
```

Plain `depends_on` only waits for the container to exist, which can leave nginx proxying to a socket that is not yet accepting connections.

Evidence

The screenshot displays the AWS Management Console for an EC2 instance. The instance is named `i-Ofce9325be0cc0adb` and is running Amazon Linux 2023 on x86_64, t3.micro. The instance is in a 'Running' state and has a public IP address of 18.119.112.99. The console shows various tabs like Details, Status and alarms, Monitoring, Security, Networking, Storage, and Tags. The instance is in a 'Running' state and has a public IP address of 18.119.112.99.

Figure 1. The free-tier instance in the EC2 console. Amazon Linux 2023 on x86_64, t3.micro, running, with public address 18.119.112.99 in us-east-2.

Verification results

Requirement	How it was verified	Result
Listens on 8000 internally	<code>docker compose ps</code> shows <code>8000/tcp</code> with no host binding	Pass
GET /api/health	HTTP 200 from the public address	Pass
POST /api/items	HTTP 200, returns <code>{"message": "added"}</code>	Pass
GET /api/items	Returns the posted items	Pass
Data stored in /data	<code>app.db</code> present, owned by <code>appuser</code>	Pass
Runs as non-root	<code>id</code> returns <code>uid 10001</code>	Pass
uvicorn without <code>--reload</code>	Container command inspected	Pass
Named volume at /data	<code>cloud-computing_appdata</code> mounted at /data	Pass
Data survives down then up	Items identical before and after	Pass
Nginx in a separate container	Two distinct containers	Pass
Only port 80 published	Only nginx has a host binding	Pass
Proxies /api/ by service DNS	<code>proxy_pass http://api:8000</code>	Pass
Port 8000 not public	<code>curl</code> times out after 8 seconds	Pass
Starts with one command	<code>docker compose up -d --build</code>	Pass

Problems encountered

Compose could not build without buildx

On a clean Amazon Linux 2023 instance the build stopped immediately:

```
compose build requires buildx 0.17.0 or later
```

Current Docker Compose delegates image building to buildx, and the Amazon Linux `docker` package does not ship that plugin. The fix was to install the matching buildx release into the same `cli-plugins` directory as the Compose plugin. The repository's setup script now does this automatically, mapping `x86_64` to `amd64` and `aarch64` to `arm64`, since the buildx release assets use different architecture names than the Compose assets.

Container name conflict on a second local clone

```
#9 [3/6] COPY requirements.txt ./
#9 CACHED

#10 [4/6] RUN pip install --no-cache-dir -r requirements.txt
#10 CACHED

#11 [5/6] COPY main.py ./
#11 CACHED

#12 [2/6] WORKDIR /app
#12 CACHED

#13 [6/6] RUN useradd --create-home --uid 10001 --shell /usr/sbin/nologin appuser && mkdir -p /data && chown -R appuser:appuser /data /app
#13 CACHED

#14 exporting to image
#14 exporting layers 0.0s done
#14 exporting manifest sha256:7f290aa2d46218601a29a9eac2ad31f934ff7ac6c-f7790818b326a8b6fe453b8 0.1s done
#14 exporting config sha256:210ed5034e07305ec87077bb950a6f6ed5a74fb736832f2b80641ddba0746dc6 0.1s done
#14 exporting config sha256:210ed5034e07305ec87077bb950a6f6ed5a74fb736832f2b80641ddba0746dc6 0.1s done
#14 exporting attestation manifest sha256:53c3391c32f5009a58f6065005e78d8c4alb6d8d77bbd8a3fa499bc1f182d59d 0.1s done
#14 exporting attestation manifest sha256:53c3391c32f5009a58f6065005e78d8c4alb6d8d77bbd8a3fa499bc1f182d59d 0.1s done
#14 exporting manifest list sha256:86b93e4fd71498aee0e0e1b87046cbdefa3cc63978c56dd09d97dd161ba6317 0.0s done
#14 naming to docker.io/library/cloud-computing-api:latest
#14 naming to docker.io/library/cloud-computing-api:latest done
#14 unpacking to docker.io/library/cloud-computing-api:latest 0.0s done
#14 DONE 0.4s

#15 resolving provenance for metadata file
#15 DONE 0.1s
[+] up 4/4
✔Image c... Built 4.7s
✔Network... Created 0.2s
✔Volume ... Created 0.0s
X Contain... Error response from daemon: Conflict. The container name "/fastapi_api" is already in use by container "200a5411cbb2ecc65ff6c44364a16afcd1ad1c0e409c067ccc0e5c35e2af3a05". You have to remove (or rename) that container to be able to reuse that name. 0.1s
Error response from daemon: Conflict. The container name "/fastapi_api" is already in use by container "200a5411cbb2ecc65ff6c44364a16afcd1ad1c0e409c067ccc0e5c35e2af3a05". You have to remove (or rename) that container to be able to reuse that name.
```

Figure 4. The image builds successfully, then startup fails because the container name is already taken.

The compose file pins fixed names with `container_name: fastapi_api`. Those names are global to the Docker daemon, not scoped to a project, so a second clone of the repository on the same machine cannot start while the first is running. Running `docker compose down` in the original directory released the names. The alternative is to drop `container_name` and let Compose generate project-scoped names.

SSH refused the private key

Connections failed with `Permission denied (publickey)` even though the key was correct. On Windows, OpenSSH refuses a private key whose file permissions allow access beyond the owner, and this key was readable by several accounts. Restricting the file to the owner alone resolved it.

Reproducing this deployment

Launch a free-tier Amazon Linux 2023 instance with inbound TCP 80 open to the world and TCP 22 open to your own address only. Do not open 8000. Then connect and run one command:

```
curl -fsSL https://raw.githubusercontent.com/Alan-911/Cloud-Computing/main/scripts/ec2-setup.sh | bash
```

The script installs Docker, the Compose plugin and buildx, clones the repository, builds and starts the stack, waits for the health check to pass, and smoke-tests all three endpoints through the proxy.

Repository contents

Path	Purpose
app/Dockerfile	Application image. Non-root, no reload, port 8000.
app/.dockerignore	Build-context exclusions. Sits in <code>app/</code> because that is the build context.
app/main.py	FastAPI application, supplied by the starter.
docker-compose.yml	Two services, one named volume, health-gated startup.
nginx/nginx.conf	Reverse proxy configuration, supplied by the starter.
scripts/ec2-setup.sh	One-command instance bootstrap.
docs/DEMO_SCRIPT.md	Commands and talking points for the live checkoff.